



krishna
securify project
May 08, 2026

8

Table of Contents

Confidentiality and Distribution Restrictions

The conclusions and recommendations in this report represent the opinions of Securify.

Determinations of appropriate corrective action(s) are the responsibility of the entity receiving the report.

This report and/or any other materials furnished by Securify in connection with this engagement is confidential and may not be duplicated, modified, or otherwise reproduced and distributed without the express prior written consent of Securify or krishna. Because this work may contain copyrighted images or other material, permission from the copyright holder may also be necessary if you wish to reproduce.

Introduction

As part of an ongoing security program, krishna identified the need to conduct an application security assessment of its Web application & APIs.

This report presents the agreed scope, methodology, risk measurement model, summarized findings, and detailed technical observations for the selected assessment window.

The following lists the objectives of this assessment:

- Determine the overall security posture of the application
- Provide a list of key findings and recommendations for remediation
- Document the assessment scope, risk evaluation, and final outcomes
- Support remediation planning with actionable technical detail
- Securify's approach to the assessment
- Assessment scope
- Key findings listed with their qualitative risk assessment
- Detailed recommendations for each finding
- A remediation plan

This report includes the following parameters and results of the assessment:

- Securify's approach to the assessment
- Assessment scope
- Key findings listed with their qualitative risk assessment
- Detailed recommendations for each finding
- A remediation plan

Approach

Securify performed a Runtime Application Vulnerability Assessment of the krishna's web application, associated APIs, and all components defined within the assessment scope.

The assessment was conducted using industry-accepted methodologies, primarily based on the OWASP Web Security Testing Guide (WSTG) and OWASP ASVS, and involved both manual testing and automated analysis.

Runtime Application Vulnerability Assessment

The Runtime Application Vulnerability Assessment involved detecting security vulnerabilities through detailed examination and testing of the application in a runtime environment. This assessment emulates an attack by a skilled adversary in a controlled setting and allows krishna to ascertain the kinds of vulnerabilities that may be realistically exploited. A Runtime Application Vulnerability Assessment includes the following phases:

- **Information Gathering** – The application was reviewed as an anonymous, authenticated, and privileged user to understand differences in access and behavior. Technologies, frameworks, APIs, and third-party integrations were also identified to support targeted testing.
- **Authentication Testing** – Authentication mechanisms were evaluated to determine the strength of login controls, password policies, and account recovery processes. Protections against brute-force attempts and session takeover scenarios were also reviewed to ensure users are securely authenticated.
- **Authorization Testing** – Tests were conducted to identify weaknesses in access control, including attempts to access other users' data or privileged functionality.
- **Session Management** – Session handling was assessed to ensure secure creation, storage, and invalidation of session tokens.
- **Input Validation Attacks** – User-controlled input fields were tested with malformed and malicious data to identify injection flaws and logic bypasses. This included attempts to exploit unexpected behavior, access unprotected functionality, or inject vulnerabilities such as XSS, SQL Injection, and Command Injection.
- **Business Logic Testing** – The application workflows were reviewed to identify opportunities to misuse or bypass intended processes. This included testing for logic errors, insufficient validation, and scenarios where typical constraints could be circumvented.

Assessment Limitation

The ever-changing technology landscape and the increasing sophistication of attacks against networked systems are reasons why no entity can truthfully claim to identify all security issues or guarantee the lifetime security of an organization's network and applications. Note that this point-in-time assessment was based on a best-effort basis. It was also performed only in the environment provided by krishna. Thus, changes to the environment may impact the applicability of the results provided herein.

Securify cannot guarantee 100% coverage for any security assessment.

Findings and Recommendation

The sections below summarize the observed risks and provide the measurement criteria used to classify findings across the engagement.

Each finding is evaluated using the same impact and likelihood model so remediation can be prioritized consistently.

Risk Classification

The remainder of this report describes the vulnerabilities that Securify identified as part of the assessment, their impact, and recommendations for resolving the vulnerabilities. To assist in determining the risk posed by these vulnerabilities, Securify leverages the OWASP Application Security Risk Rating Methodology. The observations have been categorized based on technical Impact and Likelihood, explained below. These impact and likelihood scores may be further modified by krishna based on the business criticality of the target.

Measurement of Impact

Impact is an estimation of the potential damage via a successful exploit of a vulnerability. We'll use the following factors to help qualitatively determine the impact of a vulnerability.

- Low Impact: When most/all factors indicate limited consequences (e.g., non-sensitive data, no ability to alter/delete key data, and low victim count).
- Medium Impact: When about half of the factors suggest higher damage and half point to limited effects.
- High Impact: When most/all factors highlight significant damage (e.g., sensitive data loss, wide data corruption, and a large number of victims).

Impact	Description
Low	
Medium	
High	

Measurement of Likelihood

Likelihood is a qualitative estimation of the probability of an attacker exploiting the vulnerability in question. In order to determine the likelihood of exploitation, we can consider the following factors:

- Low Likelihood: Most/all factors suggest significant barriers to exploitation (e.g., complex skillset, limited attackers, high cost, or complex delivery).
- Medium Likelihood: About half the factors indicate ease of exploitation, while the other half show barriers.
- High Likelihood: Most/all factors point to easy and accessible exploitation (e.g., basic skills, low cost, and simple attack mechanisms).

Likelihood	Description
Low	
Medium	
High	

Overall Risk

The following graph illustrates how Impact x Likelihood scores translate to overall Low, Medium, and High-risk ratings:

Overall risk is derived from the intersection of impact and likelihood and is used to prioritize remediation efforts. Findings with the highest combined rating should be addressed first, especially where exploitability and business consequence are both significant.

- **Low Impact + Low Likelihood** = Low Risk
- **High Impact + High Likelihood** = Critical Risk

Zero-risk Issues

There are cases where a particular issue doesn't have direct security impact or likelihood of exploitation. These could be issues related to documentation or semantics that are suggested based on the security assessors' technical and industry experience. Such issues shall be reported with an Informational risk rating.

Vulnerabilities

Summary

Below is a summary of the vulnerabilities discovered during the assessment, ranked in order of risk as determined:

Vulnerability	Risk
Privilege Escalation via Unrestricted API Endpoint	High

Detailed Vulnerabilities

Privilege Escalation via Unrestricted API Endpoint

Risk: High

Description:

A privilege escalation vulnerability exists where a standard user can escalate their account privileges to administrative rights. This is achieved by sending a crafted PATCH request to the '/api/user/update' endpoint. This flaw allows a lower-privileged user, such as a 'manager' role, to bypass intended access controls and perform actions reserved for higher-privileged roles, potentially leading to unauthorized administrative access, user creation, or critical system setting alterations.

Affected URL:

- <https://test.com/api/user/update>

Impact and Likelihood:

- **Impact:** The impact of this privilege escalation is severe. A successful exploitation allows a low-privileged user to gain administrative control over the application. This can lead to unauthorized access to sensitive data, modification or deletion of critical system configurations, creation of new administrative accounts, and complete compromise of the application's integrity and confidentiality. The ability to perform actions reserved for higher-privileged roles poses a significant business and security risk.
- **Likelihood:** The vulnerability is highly likely to be exploited as it involves sending a crafted PATCH request to a specific API endpoint. An attacker with a standard user account can easily construct and send such a request, given knowledge of the endpoint and the parameters required to modify user roles or privileges. The direct nature of the exploit without complex prerequisites contributes to its high likelihood.

Steps to Reproduce:

Step 1: Log in to the application with a standard user account (e.g., 'manager' role).

Step 2: Identify the '/api/user/update' endpoint.

Step 3: Craft a PATCH request to the '/api/user/update' endpoint, including parameters designed to modify the user's role or privileges to an administrative level.

Step 4: Send the crafted PATCH request.

Step 5: Verify that the standard user account now possesses elevated administrative privileges.

Recommendations:

- ****Implement Robust Access Control:**** Ensure that all API endpoints, especially those modifying user roles or sensitive data, have strict server-side access control checks. These checks must verify the user's authorization level before processing any requests.
- ****Principle of Least Privilege:**** Adhere to the principle of least privilege, ensuring that users and system processes are granted only the minimum necessary permissions to perform their intended functions.
- ****Input Validation and Sanitization:**** Implement comprehensive input validation and sanitization on all incoming requests to prevent malicious data from being processed. Specifically, validate that users cannot modify attributes they are not authorized to change.

- ****Role-Based Access Control (RBAC): **** Enforce a strong RBAC model where user roles are clearly defined, and permissions are strictly associated with these roles. Any attempt to modify a user's role should require explicit authorization and verification.

References:

- https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- <https://cwe.mitre.org/data/definitions/269.html>

Appendix A

The appendix content below is static template material and will be rendered into reports that use this template.